PICO SPACE PROGRAM 2026 · 开发者大赛

声之建筑
让声音长成建筑

PICO Spatial 赛道 · 单人零硬件预算 · 方向 A SONOSPHERE 的完整落地方案

一份可以直接开工的执行稿：作品机制、官方空间音频 API 核对结果、三处必须修正的设计错误、逐周排期与每周闸门、风险止损线、文件结构与第一周逐日拆解、路演脚本与评委问答。

01	0	60s	4
独占空间音频	**硬件支出**	**唯一素材**	**空间表面**
SPATIAL AUDIO	ZERO CAPEX	ONE RECORDING	STAGE + 3 PANEL

91	18–22	11.22
六维预估	新写人日	止损闸门

落地方案 v1.0 · 执行稿
2026-11-08 · 距提交截止 41 天
依据：官方 SDK 6.1 文档 + 已验收工程 + 竞品情报

CONTENTS · 目录

本方案书的读法

全书按「先给结论，再讲为什么，再给怎么做」的顺序编排。第 1 章是结论与三处关键设计修正，第 2 章是与竞品的正面对位，第 3–4 章是作品机制与官方空间音频 API 的核对结果，第 5–6 章是场景、交互、多窗口与内核、素材，第 7 章是硬指标、排期、风险与评分，第 8 章是可直接照着敲的文件结构与第一周逐日拆解。**目录每一行都可点击跳转。**

01 **结论与三处关键修正** 3

为什么现在做这个方向 3

02 **竞品对位：他们赢在第一眼** 4

03 **作品机制与「显影」开场** 6

建筑感的三层 6

04 **空间音频引擎（官方 API 核对）** 7

API 清单与参数 8

05 **场景 · 交互 · 多窗口** 10

06 **音频内核与素材** 11

07 **硬指标 · 排期 · 风险 · 评分** 13

08 **执行落地与决策** 14

第一周 Day 1–5 15

与方向 D 的关系 16

A **附录 A · 官方 API 与 Stage 配置速查** 18

一句话结论 声之建筑值得做，而且比 11.07 那版判断更强：**空间音频是官方点名的硬指标，而竞品三队全部没做**；它还是核心 SDK 能力（不需实验标志、不依赖 OS 6），因此真机门槛比「拍照变山」那条路更低。真正的死穴只有一个——**拿不到真机**。止损线设在 **11.22**，届时验不出声源定位就立刻切方向 D，切换成本约 1 天。

CHAPTER 01 • 结论

结论与三处关键修正

声之建筑值得做，而且比 11.07 那版判断更强。三条新依据：①**竞品三队全部没有空间音频**——官方硬指标点名的那一条，是 A 队（空间社交）、B 队（AI 校园）、C 队（机器人实验室）都没碰的位置；②**空间音频是 Spatial SDK 的核心能力，不是实验性 API**，不需要 `pico.spatial.use_experimental_api`，不要求 PICO OS 6，不依赖 SpatialML；③**零外部依赖**——不需要后端、账号、网络、模型下载、3D 资产，只需要一段 60 秒录音。

唯一真正的死穴 空间音频的听感（方向、距离、指向性）**只能在头显上验证**，模拟器只能验证「声音在播、坐标在动」。

止损线提前到 W2：若 11.22（W2 结束）仍无法在真机上验证声源定位，立即切方向 D。 理由不变且更硬：拿不到的设备不会因为再等一周就来；而 D 的全部分数都不依赖它。

1.1 为什么现在做这个方向

独占一条官方硬指标
空间音频在官方五条硬指标里点名，且三支竞品队伍**一个都没做**。这是唯一能「独占」的位置——不是做得更好，是别人根本没碰。

技术门槛反而更低
空间音频是核心 SDK 能力（`ObjectAudioComponent` / `AmbientAudioComponent` / `Directivity` / `AudioEvents`），不需要实验性标志，不依赖 OS 6。而「拍照变山」那条路依赖 SpatialML → 要求 OS 6 → PICO 4 Ultra（OS5）可能整个用不了。

媒介不可替代
声音的方向性、远近、指向性在 2D 屏幕上**不存在**。评委戴上头显转一圈就能听出空间结构——这是屏幕永远做不到的，也是「只有 XR 能做」的硬证明。

零外部依赖
不需要后端服务、账号体系、网络请求、模型下载、3D 资产库、Tripo。演示现场断网照常跑——而竞品三队全部依赖后端或 LLM，断网即崩。

评委能亲手参与
转身找声源、绕到柱背后听变化——这是「体验」而不只是「观看」。三队的最小交付都是线性点击流程。

可自证深度
作品可溯源：公开仓库 + 内核单测 + 安装包与 Commit ID 一一对应。路线为「一个人做的」这一质疑的正面答案。

1.2 与 11.07 概念稿的三处关键修正

方案书 §4 的概念稿写于 11.07。本轮逐条核对了官方 SDK 6.1 的 agent-vault 文档，发现**三处必须修正的设计错误**，其中一处是根本性的。

修正	原稿写法	核实后	为什么必须改
① **主窗形态** 根本性	24 根柱子放在半径 1.6 m 的环上，装在 **Volumetric 主窗**里	主窗必须是 **Stage 沉浸场景**（`DefaultStage`），3 个副窗维持 Planar	窗口体实测 **0.768 m 立方、X 方向可见仅 ±0.384 m**，1.6 m 的环根本放不下，只有正前方约 ±120° 露得出来且柱子会被切边。官方「Create immersive spatial audio」样例用的就是 `DefaultStage` + Planar 控制面板——**声源能绕着你走，前提是它是 Stage**

修正	原稿写法	核实后	为什么必须改
② **遮挡实现**	「绕到柱背面时该声源被遮挡变闷」——计划自己写遮挡模型	用官方内建的 `Directivity(pattern, sharpness)`	指向性是官方组件的一等参数：`pattern`（0 全向 → 1 八字形）决定极性图形状，`sharpness` 决定聚焦度。**零成本拿到「绕到背面声音真的变闷」**，而这正是「声音真的在那个位置」最省力也最难反驳的证明
③ **音画同步**	自己算播放进度，对齐柱子的点亮与呼吸	用官方 `AudioEvents`——按播放时间点触发回调	同步由系统保证，**不受帧率与调度抖动影响**。自己算进度会随负载漂移，在决赛现场掉帧时最先崩掉的就是音画同步

还有一处原稿说错了，这里更正 §4 写「影像内核的直方图 / 缺陷判据可以扔掉」——**扔的是语义，不是骨架。** `Frame → Analyzer → Field → MeshBuilder → MeshModel` 这套分层照抄给音频侧（`AudioFrame → SpectrumAnalyzer → EnergyField → ColumnMeshBuilder`），连同 42 个内核单测的测试基建一起转过去。详见第 8 章。

六维预估	新写工作量	止损闸门	外部依赖
91 / 100	**18–22** 人日	**11.22**	**1** 段录音
四方向第一，领先 D 4–5 分	不含稳定性压测与打磨	真机验不出定位即切 D	零后端 / 零账号 / 零网络

CHAPTER 02 • 对位

竞品对位：他们赢在第一眼

情报来源：飞书 wiki《X University × PICO Space Program 2026 — 三队组建与开发安排》。这不是一个竞品作品，是**一个组织的三队作战计划**：40+ 精神股东、12 名正式队员（三队各 ≤4 人）、名单外联合共建成员开放参与（设计 / 3D 美术 / AIGC / 音效 / 测试 / 视频）。

队伍	赛道	创意原型	最小交付	官方「空间音频」
A 队 XLink Spatial	**PICO Spatial (Kotlin)**	把 LinkedIn 搬进三维空间（空间社交 + AI 匹配创造者）	查看创造者 → 发现项目 → 发起连接	无
B 队 Living World	Unity Spatial	AI 原生虚拟大学（AI Scholar 导师 + X Quest 任务）	进入校园 → 与 AI 对话 → 领取任务	无
C 队 X Future Lab	OpenXR	沉浸式未来实验室（机器人 / AI / 智能硬件 + 互动挑战）	进入实验室 → 操作设备 → 完成挑战	无
我们 • 声之建筑	**PICO Spatial (Kotlin)**	**把不可见的声音变成能走进去的建筑**	**转头找声源 → 绕到柱背听变化**	最强命中

2.1 他们三个创意的共同结构：「空间化的已知品类」

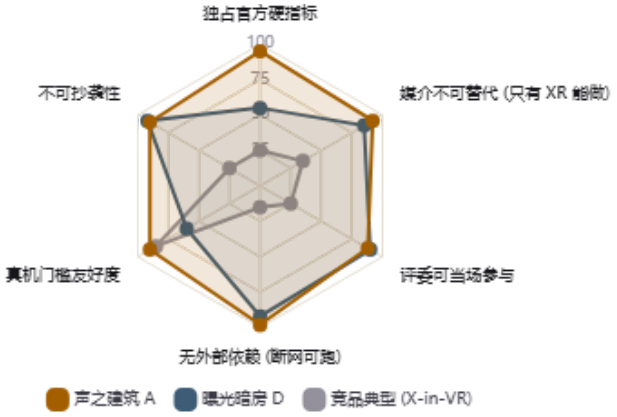
把三个方向摊开看，它们是同一类东西的不同实例：**给一个大家熟悉的 2D 品类套上 VR 外壳。**这在评审语境里是**安全牌**——评委秒懂、演示不易翻车。但它有两个死穴：

死穴一 · 没有一个是「只有 XR 能做」的
把 LinkedIn、校园、实验室搬进 VR，PCVR 或手机 VR 都能做到八成。评委问「**为什么必须戴头显**」，他们只能答「沉浸感更强」——这是 XR 评审里的弱答案，直接压「创新 25」分。

死穴二 · 抄起来很快
「社交进空间」「AI 进校园」是任何人一周能想到的提案，**没有防御性**。我们把一段录音变成一栋建筑，抄不走——因为它需要「音频分析 → 空间几何 → 空间音频定位」三件事同时成立。

竞争结构对比：五个维度上谁占优

同一把尺子量三个对象。「竞品典型」代表 X-in-VR 类作品的一般形态。



2.2 逐项对位

维度	竞品三队	我们	判定
创意原型	空间化的已知品类	不可见的物理量 → 可走进去的空间	结构不同
官方硬指标「空间音频」	三队全无	逐柱定位 + 指向性 + 距离衰减 + 混响	独占
只有 XR 能做?	否（八成可复现）	是（声音的方向性在 2D 屏上不存在）	我们赢
评委能亲手参与?	否（线性点击流程）	是（转身找声源 / 绕到柱背）	我们赢
依赖后端 / 账号 / 网络	全部依赖（匹配服务 / LLM / 任务系统）	全部不依赖，断网照常跑	我们赢
概念图 vs 实际落地	三张全是 AI 生成图，预期被顶得很高	用真实运行截图当概念图	我们稳
美术 / 音效 / 视频	各有专人（我们已决定不加入）	一个人扛：贴图自烘焙 + 空间音源自做 + H3 出片	他们赢
团队叙事	4 人团队，资源占优	单人：可能被问「一个人做的？」	他们赢

路演一句话（不点名任何队伍） 「市面上有的是 AI 驱动的空间应用。我们做的是**让声音变成一栋能走进去的建筑** —— 你绕过它背面，声音真的会变。这件事在屏幕上永远做不到。」

CHAPTER 03 • 机制

作品机制与「显影」开场

SONOSPHERE

声之建筑

听见的每一秒，在空间里长出会发声的建筑。

一段 60 秒的声音被切成 24 片，绕着你立成 24 根柱子。柱子越高，那一刻的能量越强；每根柱子都是一个真实声源，**你绕着建筑走，声音也跟着转。**

3.1 基本映射

输入 / 元素	映射	评委能读出什么
60 秒真实音频	切成 **24 片**（每片 2.5 秒）	时间有结构了——不是一条连续的流，是可以被「立起来」的 24 个瞬间
一根柱子	= 一片（24 根，等角分布成环，半径 1.6 m）	「一根柱子 = 一秒」——路演首句就说清
柱高	= 该片总能量（0.15 – 1.4 m）	响的地方高，静的地方矮 —— 建筑的轮廓就是这段声音的动态
柱色	= 频谱重心：低频深琥珀 / 中频橙 / 高频冷白	不用看数据也能感到「这段声音是沉的还是脆的」
柱面纹理	= 该片 24 频带能量的竖条（128 列 → 24 格，烘焙进贴图）	凑近看，每根柱子内部也有结构 —— 细节是「真做出来的」的信号
柱顶标签	= 时间码 / 频段 / 响度 dB	可读 = 可讨论。评委能指着某根柱问「这段是什么」
地面光环	同心光环 = 时间轴，当前播放位置那圈亮起	「现在听到的是第几秒」——空间化的进度条

3.2 建筑感的三层（缺一层就退化成「声音可视化」）

「把声音画成柱子」本身并不稀奇，任何频谱可视化都能做。**让它成为「建筑」的，是下面三层叠起来。**

层	手段	评委感知	为什么这一层不可省
几何层	Stage 沉浸场景 + 24 柱环形 + 地面声呐光环 + 暗场	「我站在一圈柱子中间」	只有 Stage 能让声源绕着你走；窗口体放不下这个环
指向层	`Directivity(pattern, sharpness)` 逐柱不同	「我绕到这根柱子后面，它变闷了」	**这是「声音真的在那个位置」最省力也最难反驳的证明**——不是音效在循环，是有方向的声源
混响层	`reverbVolume` + `AudioMixerGroupResource.setVolume()`	「整个空间的大小 / 材质在变」	同一个声音换个混响 = 换了个房间。这一层直接命中「建筑」的语义

指向层的额外收益 让**低频柱用更全向的 pattern、高频柱用更聚焦的**，评委就能听出这是两套不同的空间线索：低频包裹感强、高频定位感极强。 这是零成本的设计判断，但听起来立刻「专业」。

3.3 开场仪式：把「显影」做成一个参数

官方 Stage 有四种 style，其中 **Progressive** 可以用 `immersion`（0–100）连续调节「真实环境透传」与「虚拟场景」的比例：

```
immersion = 0    →  几乎纯真实房间，虚拟实体不可见
immersion = 50   →  真实环境与虚拟场景各占一半（真实环境的光照仍来自 VST）
immersion = 100  →  等价于 Full：纯虚拟场景，真实环境被完全遮住
```

这就是「显影」本身　immersion 从 0 推到 100：你眼前的客厅像一张照片在显影液里慢慢浮出，变成由声音构成的建筑。

不需要任何模型、贴图或动画资产——只需要一个参数在 6 秒内推过去。这是零资产成本、零外部依赖、且在模拟器上就能验收的戏剧性开场。

manifest 配置项	取值	说明
`pico.spatial.stage.id`	SONOSPHERE	默认 Stage 的唯一名
`pico.spatial.stage.style`	待实测	Progressive 对应的数字枚举 —— 文档只给了名字（Automatic / Mixed / Progressive / Full），**没给数字**。W1 第一天必须四个取值各试一遍
`pico.spatial.stage.immersion`	50	默认沉浸度；开场动画从 0 推到 100
`pico.spatial.stage.immersion_min`	0	下限
`pico.spatial.stage.immersion_max`	100	上限
`pico.spatial.stage.upperlimb`	1	上肢可见 —— 交互时手在画面里是加分项

退路（必须提前想好）　若 Progressive 在模拟器上不可用或表现不对，退路是 **Mixed + 自做一个内翻的暗色天球**盖住 VST，再用贴图与自发光压出暗场。视觉上仍有「暗房 / 显影」的语汇，只是失去「真实房间被吞掉」那一层戏剧性。**这条退路不依赖任何新 API。**

CHAPTER 04 · 引擎

空间音频引擎（官方 API 核对）

本章每一个 API 名与参数都来自本机 PICO SDK 6.1 的 `agent-vault` 官方文档（`com.pico.spatial.core.ecs.audio` / `com.pico.spatial.audio`），**不是凭记忆写的。**底层用 **HRTF** 渲染：「models the acoustic characteristics of the human ear based on HRTF」。

为什么这一章最重要　空间音频是**核心 SDK 能力**，不是实验性 API —— 不需要 `pico.spatial.use_experimental_api`，**不要求 PICO OS 6**，不依赖 SpatialML。 这意味着它的真机门槛比「拍照变山」那条路低得多（那条要 OS 6，4 Ultra 是 OS 5）。

4.1 API 清单与参数

API	包	用途与关键参数
`AudioResource.load(name, "asset://xxx.wav", LoadType.FROM_ASSETS)`	core.ecs.audio	从 assets 加载音频资源
`ObjectAudioComponent`	core.ecs.audio	**声源绑实体**：`volume`（总音量）、`Directivity(pattern, sharpness)`（指向性）、`distanceAttenuationMode`（距离衰减）、`reverbVolume`（混响音量）。位置或朝向变化时自动调整输出
`Directivity(pattern, sharpness)`	core.ecs.audio	`pattern` = 0 全向 → 0.5 心形 → 1 八字形；`sharpness` = 聚焦程度。**绕到柱背面声音变闷就是它**，官方内建，不用自己做遮挡
`DistanceAttenuationMode`	core.ecs.audio	`INVERSE_SQUARED`（默认，按距离平方反比衰减）/ `FIXED`（恒定）。**柱阵缩放时能听出变远变近**
`AmbientAudioComponent`	core.ecs.audio	环境床（房间底噪）—— 让「空间」听起来不空
`ChannelAudioComponent`	core.ecs.audio	通道音（副窗反馈音等）
`entity.prepareAudio(resource)` / `entity.playAudio(resource)`	core.ecs.audio	拿到 `AudioPlayerController`，控制 `play()` / `pause()` / `stop()`
`AudioEvents`	core.ecs.audio	**按播放时间点触发自定义回调** ⇒ 音画同步交给系统，不自己算进度
`AudioMixerGroupsComponent` + `AudioMixerGroupResource.setVolume()`	core.ecs.audio	统一控制一组资源的音量 / 混响（做交叉淡入淡出、避免爆音）
`AudioResourceLibraryComponent`	core.ecs.audio	以字典方式批量增删查 24 份资源
`AudioGroupResource` + `AudioGroupResourcePlayMode`	core.ecs.audio	资源集合按播放模式随机选一个 —— 做「随机环境声」很好用
`AudioResourceConfig(mixerGroupID, ambisonicsType, ...)`	core.ecs.audio	资源配置，**含 AmbisonicsType**（360° 音频）
`SpatialAudioTrackExtension`	spatial.audio	给第三方播放器接空间音频：`SpatialAudioMode.OBJECT / AMBIENT`，支持 `attachToEntity`（绑 3D 物体）或 `attachToContainer`（绑窗口）

4.2 核心代码骨架

柱子实体**就是**声源 —— `ObjectAudioComponent` 直接挂在柱子的 Entity 上。这样柱阵旋转 / 缩放时，**声音位置自动跟着走**，不需要任何额外同步代码。

```
// 柱子 = 声源
val column = Entity()
column.components.set(
    ObjectAudioComponent(
        volume = 1.0f,
        Directivity(pattern = 0.35f, sharpness = 0.60f),
        distanceAttenuationMode = DistanceAttenuationMode.INVERSE_SQUARED,
        reverbVolume = 0.5f
    )
)
val player = column.prepareAudio(audioResource)   // AudioPlayerController
player.play()
```

官方文档给出的等价写法（第三方的 `AudioTrack` / `ExoPlayer` / `MediaPlayer` 也能接）：

```
val extension = SpatialAudioTrackExtension()
extension.spatialAudioTrackExtensionConfig(
    mode = SpatialAudioMode.OBJECT,
    isAmbisonic = false,
    builder = audioTrackBuilder
)
extension.attachToEntityWithBuilder(targetEntity, audioTrackBuilder)
val audioTrack = audioTrackBuilder.build()
```

4.3 声源策略

24 柱 = 24 个潜在声源
但**同时只激活能量最高的 4 个**。理由有两条：一是性能（24 路空间化渲染 + 衰减计算）；二是听感清晰（同时响 24 个会糊成一片，评委反而听不出方向）。

声源实体即柱子
`ObjectAudioComponent` 挂在柱 Entity 上 ⇒ 柱阵变换时声音位置自动跟随，无需同步代码。

资源统一管理
24 段音频用 `AudioResourceLibraryComponent` 管理；播放头推进靠 `AudioEvents`。

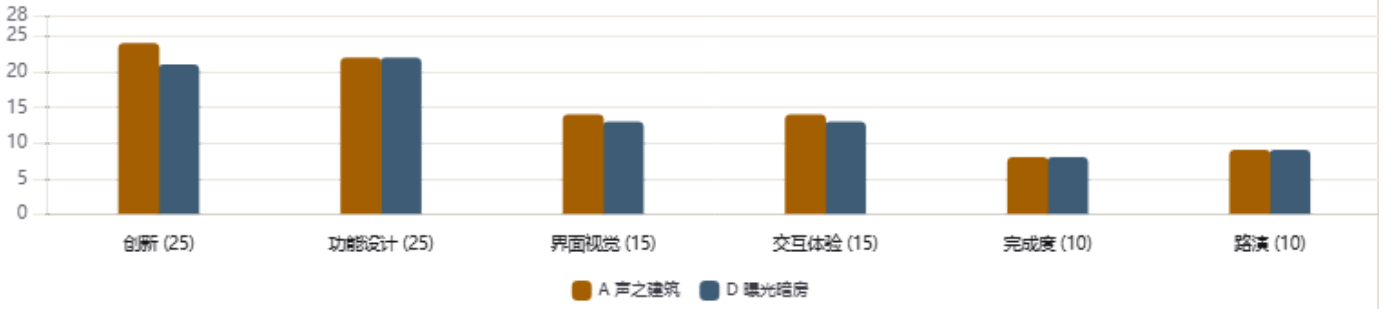
切换要交叉淡化
换声源时用 `AudioMixerGroupResource.setVolume()` 做淡入淡出，否则会出现爆音 —— 爆音在评委的耳朵里是致命的。

不做实时麦克风采集
输入是文件回放。理由：模拟器无麦克风；回放可复现、可调试、可在提交包内自洽（不依赖任何授权）。

频段决定指向性
低频柱 pattern 更全向（包裹感），高频柱 pattern 更聚焦（定位感）。零成本，但听起来立刻专业。

六维预估：声之建筑 vs 曝光暗房

声之建筑的优势集中在「创新」与「交互体验」两维 —— 前者来自独占空间音频硬指标，后者来自评委可亲身参与。



官方样例工程是最快的启动路径　官方「**Create immersive spatial audio**」样例的完整结构是：`DefaultStage` + Planar 控制面板 + 环境音 + 移动声源，源码在 `app/src/main/java/com/pico/spatial/sample/spatialaudio/`：`Main.kt`（声明 Stage 与控制面板）/ `ui/ImmersiveScene.kt` / `ui/AudioControlViewModel.kt` / `data/AudioResourceStore.kt` / `manager/AudioPlayer.kt` / `ecs/FlyTrajectory.kt`。

进官方群时务必问一句：能否提供这个样例工程。拿到它等于拿到本方向的地基。

CHAPTER 05 • 场景

场景 · 交互 · 多窗口

5.1 场景与视觉系统

项	规格	为什么这么定
场景	**Stage 沉浸场景**（不是窗口），style = Progressive，开场 immersion 0→100 推满	只有 Stage 能让声源绕着你走
柱体	八棱柱（比圆柱便宜，棱面在暗光下更出体积），底面固定在半径 1.6 m 的圆上	八棱柱有明确棱线，暗场里轮廓更硬朗
柱高	0.15 – 1.4 m，随该时间片能量线性映射	能量差异要肉眼可辨，否则「建筑」变成「栅栏」
配色	频段着色：低频深琥珀 / 中频橙 / 高频冷白。**明暗与光照烘进贴图**	顶点色与方向光对自定义网格无效（已实测），必须烘焙
柱面纹理	24 频带能量竖条（128 列 → 24 格）	凑近看有细节 —— 「真做出来的」的可信信号
地面	半径 2.2 m 暗色圆盘，烘一圈同心光环（声呐），当前播放位置那圈亮起	空间化的进度条，比平面进度条更直觉
环境	暗场压到 12%，只有柱体与光环自发光	空间音频的听感依赖「没有别的东西分散注意力」

视觉上的战略选择　竞品三队的概念图全是**高亮玻璃面板 + 浅灰 + 青绿**（AI 出图最泛滥的风格）。**我们反着来** —— 深墨 + 琥珀 + 暗场。 这是四份材料里唯一的暗色系，辨识度直接拉满；而且暗场是空间音频听起来「就在耳边」的前提，审美与技术在这里是同一件事。

5.2 交互清单（每条写死「可见反馈 + 降级路径」）

规则：**每个动作都必须有立刻可见的变化**，且**每个动作都必须有不用手势的降级路径**。手势在真机验收之前，模拟器只保证「不崩、能自动演出」。

交互	动作	可见反馈（缺一不可）	降级路径
聚拢	五指捏合 / 双手合拢	柱高压到 20%、半径收到 0.6 m、光晕收缩 —— **形变必须肉眼可辨**，不能只换颜色	主窗底部按钮「聚拢 / 展开」
划过	手指横切过某根柱	该柱亮起 + 其余 23 根降到 25% 亮度 + 顶部弹出标签（时间 / 频段 / 响度）	副窗 1 频谱环上点对应扇区
转盘	单手绕圈拖动	整个环旋转，柱子擦过你的位置；**经过正前方的柱子时该柱音轨被提亮**	副窗 3 的方向拨盘
缩放	双手开合	环半径在 0.8–2.4 m 之间连续变化，柱距同步改变；**距离衰减随之变化（能听出变远变近）**	副窗 3 滑杆
长按	按住柱顶 0.6 s	柱体炸开成 24 个小方块（频段分块），每块悬浮成一个可读标签	副窗 2「ROOM TONE」列表

自动演出模式（强制要求）　无人操作时，柱阵必须自己动：启动后自动播放、自动按能量呼吸、自动缓慢旋转 360°（每 20 秒一圈）、自动轮换 4 个活跃声源。 **评委戴上头显的那一刻，画面不能是静止的。** 这一条与「启动即自动铺开四窗」是同一条工程原理：**演示不依赖任何一次点击。**

5.3 多窗口设计（Stage + 3 Planar）

窗口	形态	内容	作用
Stage • SONOSPHERE	**Stage**	环形柱阵 + 地面光环 + Progressive 显影开场	作品本体
副窗 1 • SPECTRUM RING	Planar	24 段频谱环，实时对应当前播放位置的能量	把「听觉」翻译成「视觉」，也是副窗 3 的锚点
副窗 2 • TIMELINE	Planar	60 秒波形时间轴 + 24 个时间片标记 + 文字转录	让「这一分钟」有可读的时间结构
副窗 3 • ROOM TONE	Planar	声源列表（哪根柱在响、多响）、混响参数、指向性参数、开关	把空间音频「工程化可见」，命中功能设计维度

四个独立空间表面
Stage + 3 个 WindowContainer，共享 `SessionBus`（已有工程，零改动复用）。官方硬指标「多窗口」不只是及格线，是评分维度里的交互体验基础。

副窗不是装饰
三层信息 = 频谱（听觉的视觉翻译）/ 时间（可读结构）/ 声学（工程参数可见）。这是「功能设计 25 分」的抓手。

Stage 的额外好处
沉浸场景可以把真实房间纳入构图 —— Progressive 推到 100 之前，观众是在自己的客厅里看柱子长出来。

CHAPTER 06 • 内核

音频内核与素材

6.1 音频内核：与影像内核同构

新建 `app/audio-core/`——**纯 Kotlin、零 Android 依赖**，与 `imaging-core` 一样可在 PC 侧跑单测。分层照抄影像内核的写法：

```
AudioFrame (PCM)  →  SpectrumAnalyzer  →  EnergyField(24×24)
                  →  ColumnLayout  →  (Stage 实体 + 声源)
```

层	做什么	输出与验收
`AudioFrame`	PCM 帧容器（样本率 / 声道 / 时间戳）	与影像内核的 `Frame` 同构
`SpectrumAnalyzer`	分帧 + 梅尔频谱	60 s → 24 段，每段 24 个频带 ⇒ **24 × 24 能量矩阵**
`EnergyField`	每段特征量提取	总能量 / 频谱重心 / 瞬态密度 / 高频占比 / **房间感指标**
`RoomTone`	房间感指标计算	频谱平坦度 + 高频衰减斜率的组合。混响多、空间大的录音给出不同的值 ⇒ 直接映射到 `reverbVolume` 与柱体「材质」

单测（≥12 条，镜像影像内核的测试风格） 静音输入全零 · 纯正弦的重心正确 · 脉冲串的瞬态密度正确 · 削波输入不越界 · 立体声左右不平衡的处理 · 时长边界（不足 / 超出 24 片）· 分段边界不重复不丢样 · 矩阵维度恒定 ……
这是「完成度 10 分」与原创性护栏的共同证据 —— 公开仓库里能跑绿的测试，比任何自述都有说服力。

6.2 素材清单：需要 hommi 提供什么

#	素材	要求	截止
1	**60 秒音频（主）**	手机录即可。**首选：朗读一段文字** —— 语速自然、有呼吸、有情绪起伏，柱子会长得不一样	11.12
2	**60 秒音频（备 A）**	城市街道（车、人、雨、施工）	11.12
3	**60 秒音频（备 B）**	雨夜窗边	11.12
4	**可选：对应的文字稿**	朗读类必给，显示在副窗 2。文字让「声音」有了语义，评委更容易记住	11.12

6.3 录音指导（决定成败）

✅ **要有强起伏**
说话、脚步、开关门、远处雷声 —— 起伏越大，柱高差异越大，视觉越有故事

✅ **在真实空间里录**
客厅 / 走廊 / 楼道 —— 空间混响本身就是我们要分析的「房间感指标」

❌ **不要纯音乐**
音乐频谱过于平稳，柱子会长得一样，视觉就没戏

❌ **不要持续噪声**
雨声、空调声全程一样 ⇒ 24 根柱子几乎等高，「建筑」变成「栅栏」

📱 **设备不限**
手机录即可。要求只有两条：**时长 60 秒**、**没有背景音乐**

🔒 **合规**
只用自己录的声音。街道录音避免拍到可识别人脸与清晰对话

✓ **零外部素材**——无模型下载、无 3D 资产、无第三方图库、无 Tripo API。这是本方向相对「指尖造物」的最大优势：**没有拿不到就做不完的环节。**

6.4 从音频到建筑的完整数据链

阶段	数据形态	落到画面 / 声音上的东西
PCM	60 s × 采样率 × 声道	原始素材
分帧	24 片 × 2.5 s	24 根柱子的存在
梅尔频谱	24 × 24 能量矩阵	柱高（行能量）+ 柱色（列重心）+ 柱面纹理（整行）
特征量	重心 / 瞬态 / 房间感	柱色、材质、`reverbVolume`、`Directivity` 的 pattern
音频事件	时间点回调	当前柱点亮、地面光环推进、活跃声源切换
空间音频	HRTF 渲染	你转头时，声音真的从那个方向来

这张表就是路演的技术骨架 评委问「这到底是怎么做的」，答案就在这张链路上：**一段真实录音 → 24×24 频谱矩阵 → 24 根柱子 → 4 个 HRTF 声源。** 每一步都可验证、可复现、不可被「AI 生成的通用方案」四个字盖过去。

CHAPTER 07 · 落地

硬指标 · 排期 · 风险 · 评分

7.1 官方硬指标命中表

官方硬指标	命中	兑现方式
多窗口 基本要求	强命中	Stage + 3 个 Planar 窗口，共享 SessionBus；副窗不是装饰，是频谱 / 时间 / 声学三层信息
3D 内容	强命中	24 根八棱柱 + 地面圆盘，自拼 MeshModel，环形建筑有真实体积
空间交互	命中	聚拢 / 划过 / 转盘 / 缩放 / 长按 5 组，每组都有可见形变 + 按钮降级 + 自动演出模式
空间音频	最强命中	逐柱 ObjectAudioComponent 定位 + Directivity 指向性 + 距离衰减 + reverbVolume + AudioEvents 音画同步。**竞品三队全部没做这一条**
MR	强命中	Stage style = Progressive，immersion 0→100 把真实房间「显影」成建筑 —— 三者中唯一真·混合现实的表达

7.2 六周排期（单人，11.08 – 12.19）

任务	W1 11.08–14	W2 11.15–21	W3 11.22–28	W4 11.29–12.05	W5 12.06–12.12	W6 12.13–19
Stage 改造 DefaultStage + manifest 四项 meta-data + style 枚举实测	■	■				
音频内核 audio-core：解码 + 24×24 梅尔频谱 + 房间感（带单测）	■	■				
柱阵 网格生成 + 高度/配色/纹理烘焙 + 显影开场	■	■	■	■		
空间音频 ObjectAudioComponent × 4 + Directivity + AudioEvents		■	■			
三副窗 频谱环 / 时间轴 / 声学参数 + SessionBus		■	■	■		
手势 + 自动演出 5 组交互与全部降级路径			■	■		
门槛补齐 30 分钟稳定性压测 + 交包脚本 + APK sha256 ↔ Commit	■					
交包闸门 11.25 通道开即交完整可安装包			■			
打磨 / 开源 / 路演稿 / 演示片			■	■	■	■

排期说明 **W1 把「30 分钟稳定性压测」与「交包脚本」放进去，是因为这两件事与方向选择无关**，先做完就不用返工。
空间音频放在 W2，是因为它依赖 Stage 里柱子的最终坐标 —— 坐标不定下来，声源就没地方挂。
11.25 交包闸门在 W3：通道一开就交完整可安装包，之后持续更新到 12.19 09:00 锁版。

7.3 风险表

风险	概率	影响	对策与止损线
拿不到真机 / 11.22 前验不出空间音频听感	中	高	W1 即向官方群索取设备型号、预约窗口与空间音频样例包；**止损：11.22 切方向 D**
Stage 改造影响现有四窗口工作台	中	中	副窗不动，只换默认容器；改完立刻回归「四表面同框」自检

风险	概率	影响	对策与止损线
`stage.style` **数字枚举与文档不符**	中	中	W1 第一天把 4 个取值各试一遍；Progressive 不可用则退到 Mixed + 暗色天球
24 个声源性能 / 音频延迟	低	中	同时只激活 4 个；W2 用合成音频先压测
手势真机失效	中	中	每组交互都有按钮 / 副窗降级；自动演出模式保证「不操作也在动」
录音素材平淡，柱子长得一样	中	中	录三条不同类型对比；朗读类自带语义起伏，最保险
评委看不懂「柱子代表什么」	中	中	副窗 1 频谱环做一一对应映射；路演首句就说清「一根柱子 = 一秒」
音频文件授权 / 隐私	低	低	只用 hommi 自己录制的声音；街道录音避免可识别人脸

7.4 六维评分预估

维度	满分	预估	判断依据
创新	25	**24**	「声音即建筑」概念清晰；**且是四方向里唯一命中竞品全缺硬指标的。**扣分点是概念本身仍属「把 A 映射成 B」型创新
功能设计	25	**22**	频谱 / 时间 / 声学三层信息 + 5 组交互 + 可调指向性 / 混响 / 衰减，闭环完整
界面视觉	15	**14**	暗场 + 环形柱阵 + Progressive 显影开场 + 频段配色，四份材料里唯一暗色系，辨识度强
交互体验	15	**14**	有身体参与、有指向性惊喜、能听见空间变化；扣分点是手势真机未验
完成度	10	**8**	四表面 + 内核 + 工具链齐备；扣分点是真机未验、稳定性未跑
路演	10	**9**	一句话可说清、10 秒有反应点、**音频自证「在现场」**
合计	**100**	**91**	**四方向第一，领先 D 4–5 分。**仍要诚实：91 是上限估计，真机未验前应按 **85–88** 折算

CHAPTER 08 • 执行

执行落地与决策

8.1 工程复用 vs 重做

类别	内容
直接复用 零改动	四窗口机制 + `SessionBus` + 贴图烘焙管线 + `MeshResource` 打包流程 + 模拟器工具链（pico-cli / adb / 截图）+ APK 构建安装链路 + 交包脚本 + **42 个内核单测的测试基建与写法**
骨架复用 换实现	`imaging-core` 的**架构骨架**：`Frame` → `Analyzer` → `Field` → `MeshBuilder` → `MeshModel`。音频侧照抄这套分层：`AudioFrame` → `SpectrumAnalyzer` → `EnergyField` → `ColumnMeshBuilder`。**§4 原稿说「可以扔掉影像内核」说重了——扔的是语义，不是骨架**
需要新写	音频解码 + 梅尔频谱分析器 + 柱阵网格生成器 + `ObjectAudioComponent` 接入 + `AudioEvents` 同步 + 三个副窗的新内容
不再需要	影像内核的直方图 / 曝光缺陷判据（曝光山脉、缺陷语义色、地形 hillshade 全部停用）
工作量估算	新写 **18–22 人日**（不含稳定性压测与打磨）。6 周 = 30 个工作日，缓冲 **8–12 天**

8.2 目标文件结构与第一周逐日拆解

```
app/
├── audio-core/                          ← 新增（纯 Kotlin，零 Android 依赖）
│   └── src/main/kotlin/com/spatialcoach/audio/core/
│       ├── AudioFrame.kt                ← PCM 帧
│       ├── SpectrumAnalyzer.kt          ← 梅尔频谱 + 24 段能量矩阵
│       ├── EnergyField.kt               ← 24×24 矩阵 + 每段特征
│       └── RoomTone.kt                  ← 房间感指标
├── app/src/main/java/com/spatialcoach/sonosphere/
│   ├── platform/                        ← SpatialApplication / LaunchActivity（Stage 改造）
│   ├── stage/                           ← Stage 内容
│   │   ├── SonosphereStage.kt           ← DefaultStage { ... }
│   │   ├── ColumnRing.kt                ← 24 柱 ECS 实体 + 柱阵变换
│   │   ├── ColumnMeshBuilder.kt         ← 八棱柱网格
│   │   └── BakedTexture.kt              ← 频段配色 + 频带条纹 + 明暗烘焙
│   ├── audio/
│   │   ├── SpatialAudioDirector.kt      ← 4 个 ObjectAudioComponent 声源调度
│   │   ├── AudioEventBridge.kt          ← AudioEvents → 视觉回调
│   │   └── AudioLibrary.kt              ← AudioResourceLibraryComponent + 24 资源
│   ├── content/                         ← 三个副窗（频谱环 / 时间轴 / 声学参数）
│   └── interaction/                     ← 5 组手势 + 降级路径
└── app/src/main/assets/audio/           ← hommi 的 60 秒 wav（切成 24 段）
```

日程	做什么	产出	完成判据
Day 1	Stage 改造：manifest 四项 meta-data + mainApp 里 `DefaultStage { }`；**把 4 个 `stage.style` 取值各试一次**	Stage 能起、能看到一个立方体	模拟器截图确认 Stage 已开；记录 Progressive 对应的数字
Day 2	`audio-core`：音频解码 + 分帧 + 24×24 梅尔频谱 + 特征量（带单测）	24 × 24 能量矩阵	单测覆盖「静音输入全零」「纯正弦重心正确」
Day 3	柱阵：`ColumnMeshBuilder` + 24 柱环形布局 + 高度映射	能看见一圈高低不同的柱子	柱高与矩阵数值一一对应（打印校验）
Day 4	贴图烘焙：频段配色 + 频带条纹 + 明暗；地面声呐光环；显影开场参数	定稿截图	截图里无灰白块、明暗层次可辨、immersion 动画可见
Day 5	空间音频接入：`ObjectAudioComponent` × 4 + `prepareAudio` + `AudioEvents`	4 个声源可定位	日志能打印每个声源坐标与 volume；能听到声音在播

Day 1 额外任务（不能省） 进官方群问 6 问（设备型号 / Kotlin SDK 能否跑 4 Ultra / 预约窗口 / 决赛是否提供头显 / 能否兼提名两奖 / 空间音频示例包），**并加问「官方『Create immersive spatial audio』样例工程能否提供」** —— 那个样例是本方向最快的启动路径。

8.3 验收命令（沿用现有链路）

```
$env:JAVA_HOME='E:\software\Dev\androidstudio\androidStudio\jbr'
$gradle='G:\minimaxworkspace\pico-spatial-2026\prototype\gradle-dist\gradle-8.13\bin\gradle.bat'
& $gradle -p 'G:\minimaxworkspace\pico-spatial-2026\app' --no-daemon :audio-core:test :app:assembleDebug --
console=plain

$env:PICO_HOME='C:\Users\11574\AppData\Local\PICO\sdk'
$cli='G:\minimaxworkspace\pico-spatial-2026\tools\node_modules\.bin\pico-cli.cmd'
& $cli app install 'G:\minimaxworkspace\pico-spatial-2026\app\app\build\outputs\apk\debug\app-debug.apk' -r
& $cli app launch com.spatialcoach.sonosphere
```

8.4 与方向 D 的关系（决策用）

维度	A 声之建筑	D 曝光暗房
独占的官方硬指标	**空间音频（三队全缺）**	无独占（MR 可用但弱）
真机 / 硬件门槛	**核心 API，不依赖 OS 6**	拍照钩子依赖 SpatialML → OS 6 → 4 Ultra 可能不行
外部依赖	1 段 60 秒音频	6–10 张照片（含 ≥3 张严重过曝）
可当场参与	转身找声源 / 绕到柱背	拍照 → 变山
六维预估	**91**（折算 85–88）	86–87
内部一致性	音频内核（`audio-core`）	影像内核（`imaging-core`，已完整）

决策建议

按 A 执行，11.22 设硬止损闸门。

理由：A 同时在「独占硬指标」和「硬件门槛」两条上占优，而这两条正是 D 的软肋。且 A 的止损线**可判定**（真机验不出定位就切），D 的止损线模糊。

若 11.22 真机验证失败：切 D。此时 D 的现成资产（曝光山脉 + 四窗口 + 42 单测 + 提交历史）全部还在，**切过去是 1 天的成本，不是 3 周**。这个「先 A 后 D」的顺序是风险最低的。

8.5 路演脚本（30 秒版）

时间	动作与台词
0–3 s	「这是一段真实录音。」
3–6 s	（immersion 从 0 推到 100）**「这是我家客厅 —— 现在它在显影。」**
6–12 s	24 根柱子从地面升起，柱子随声音呼吸。**「60 秒，被切成了 24 根柱子。一根柱子是一秒。」**
12–18 s	4 个声源同时响，评委**不由自主转头。「你现在听到的方向，是真的方向。」**
18–24 s	（走到某根柱背后）声音变闷。**「我绕到它背后 —— 你听，它变了。」**
24–30 s	（转向评委）**「声音不是被播放的，是被建造的。」**

8.6 评委最可能问的 5 个问题

问题	回答要点
这是真录音吗?	是，自己用手机录的。**刻意保留底噪与手机麦克风的质感**——这是「真实」的证据
声音定位是 SDK 自带还是你自己做的?	SDK 内建（HRTF 渲染 + `ObjectAudioComponent`）。**我做的是「把声音映射成建筑」这一层**——分析、分段、生成几何、挂声源
这套能用在游戏里吗?	能，`ObjectAudioComponent` 本来就是给游戏用的（官方文档的 VR 游戏用例就是这个）。**我做的是「让声音变成可走进去的空间对象」**
你一个人做的?	是。**作品可溯源：公开仓库 + 内核单测 + 安装包与 Commit ID 一一对应**（评分本来就以安装包 + Commit ID 为依据）
和普通声音可视化有什么区别?	普通可视化是**看**频谱；这个是**走进去**，而且绕到背面声音真的变——屏幕做不到

8.7 立刻要做的三件事

1 **今天** 进官方群问 6 问 + 加问「官方空间音频样例工程能否提供」+ 问 **PICO 4 Ultra 能否跑 Spatial SDK** 与预约窗口。**这是当前最高优先级的未知。**

2 **11.12 前** 录 3 条 60 秒音频（朗读 / 街道 / 雨夜），**朗读那条优先**——有语义起伏，柱子才不一样

3 **11.20 前** 约一次真机（半小时也行）—— 只要能戴上头显听到「绕到柱后声音变闷」，这个方向就锁定了

8.8 提交前 48 小时检查清单

锁版前逐项销账。**官方正式评分以「安装包 + 对应 Commit ID」为依据**，因此第 2 项与第 5 项是硬性的 —— 缺任何一条都会直接影响可评分性。

#	检查项	状态	判据
1	**安装包可安装**	☐	卸载后重装成功，模拟器可启动并进入 Stage；无崩溃日志
2	**APK sha256 ↔ Commit ID 一一对应**	☐	交包脚本输出对照表，提交包内含该表
3	**README 完整**	☐	作品名 / 一句话 / 操作说明 / 技术栈 / 开源链接 / 素材来源
4	**30 分钟连续运行无崩溃**	☐	准入门槛第 3 条。**附完整 log 与时间戳**（这项是硬门槛，不通过则作品根本不进正式评分）
5	**四个空间表面同框**	☐	Stage + 3 副窗同时出内容；含一张可做概念图的真实截图
6	**演示视频就位**	☐	60 秒内，含「显影开场 → 柱阵升起 → 转向找声源 → 绕到柱背」四拍
7	**报名信息核对**	☐	赛道 PICO Spatial / 组长 = 本人 / 正式名单 ≤4 人 / 联合共建者署名
8	**提交物合规**	☐	不含公司代码、ISP 参数表、内部文档、他人素材；模型与资产来源可追溯
9	**素材授权自洽**	☐	音频为本人录制；街道录音无可识别人脸与清晰对话
10	**仓库干净**	☐	无大文件、无构建产物、无密钥；提交信息逐字节无 BOM / 无 U+FFFD / 无 CRLF
11	**空间音频真机复测**	☐	绕到柱背面能听出变化；转头能定位；混响参数可听出差异
12	**锁版时间**	☐	12.18 内部验收完成，12.19 09:00 前留出 2 小时缓冲

8.9 交付节奏

11.25 首版交包 — APK + Commit 对照表 + README

11.28 功能闭环 — 三副窗 + 5 组交互 + 自动演出

12.06 开源与溯源 — 公开仓库 + 内核单测全绿

12.12 路演与演示片 — 30 秒稿 + 3 分钟稿 + 60 秒片

12.18 内部验收定版 — §8.8 清单 12 项逐条销账

12.19 锁版提交 — 09:00 截止，留 2 小时缓冲

交付物	形式	完成时点	去向
可安装包	APK（debug + release）	11.25 / 12.18	赛事提交通道
安装包 ↔ Commit 对照表	脚本输出（sha256 + commit id + 时间）	随每次交包	提交包内 + README 附录
README / 公开仓库	Markdown + GitHub（内核 + 单测）	11.25 / 12.06	提交包内 + 开源加分
演示视频 / 概念图	MP4 60 秒内 + 真实运行截图	12.12 / 12.05	提交材料 + 路演备用

APPENDIX A • 速查

官方空间音频 API 与 Stage 配置速查表

下面这张表是执行期最常翻的一页：**每一行都对着官方 SDK 6.1 agent-vault 文档逐条核对过**，可以直接照着写，不需要再去翻文档。空间音频属于核心 SDK 能力，**不需要实验标志，也不依赖 PICO OS 6**（那条要求只针对 SpatialML）。

API / 组件	类别	关键参数	在本方案里的位置
`ObjectAudioComponent`	组件	挂在 Entity 上	柱阵即声源 —— 柱子动了声音自动跟着走，不写同步代码
`Directivity(pattern, sharpness)`	参数	逐柱指向性	指向层：「绕到柱背声音变闷」—— 最省力也最难反驳的证据
`reverbVolume`	参数	混响强度	混响层：同一段声音换个混响，就是换了个房间
`AudioEvents`	API	按播放时间点回调	音画同步：柱高、光环、当前播放位置全部对齐
`AudioResource.load(…)`	API	加载音频资源	60 秒母带切成 24 片，24 根柱子各挂一段
`entity.prepareAudio`	API	预加载	开场前先备好 —— 播第一句时不卡，这是第一印象
`AmbientAudioComponent`	组件	环境音	房间底噪：让空间听起来不空
`ChannelAudioComponent`	组件	通道音	低频铺底与连续层，负责把 24 片连成一段
`AudioMixerGroupsComponent`	组件	分组音量	混响层独立调，不与主声串音
`DistanceAttenuationMode`	枚举	`INVERSE_SQUARED` / `FIXED`	柱阵半径 1.6 m 的听感校准（W2 真机调参）

另有两条备选通路，需要时再上：`AudioResourceConfig(ambisonicsType)`（全景声，用于环境层）与 `SpatialAudioTrackExtension` + `SpatialAudioMode.OBJECT / AMBIENT`（接第三方播放器）。

A.1 Stage manifest 配置项

manifest 键	取值	说明
`pico.spatial.stage.id`	`SONOSPHERE`	默认 Stage 的唯一名
`pico.spatial.stage.style`	数字枚举待实测	文档只给了名字（Automatic / Mixed / Progressive / Full），没给数字。**W1 第一天必须四个取值各试一遍**
`pico.spatial.stage.immersion`	50	默认沉浸度；开场动画从 0 推到 100，用 6 秒
`pico.spatial.stage.immersion_min` / `_max`	0 / 100	沉浸度上下限

manifest 键	取值	说明
pico.spatial.stage.upperlimb	1	上肢可见 —— 交互时手在画面里，是加分项

A.2 模拟器能验什么、验不了什么

环节	模拟器上的实际情况	怎么办
空间音频听感	验不了 —— 没有真实声场与 HRTF 听感	只能靠 **11.20 那次真机**；这是止损线的唯一依据
声源指向性	听不出「绕到柱背变闷」	真机才能证明声音真的在那个位置，别的都替代不了
混响强度	验不了	真机调参；模拟器上只调数值，不做判断
麦克风 / 实时采集	无麦克风	本方向**只播放不采集**，素材离线录好再打进包
相机与深度	无真实相机、无深度图	与本方向无关 —— 那是方向 D 的前置条件
点击与手势	`adb shell input tap` 无效、`adb root` 被拒	模拟器上根本点不动，交互验证全部推给真机
空间 UI 拾取	只认 `HMD_POINTER_SELECT_INTERACTOR` / `POINTER_HOVER_INTERACTOR` / `HAND_POKE_HOVER_INTERACTOR`	**不要写普通触摸分支**，那样在设备上也是死的
窗口体尺寸	0.768 m 立方，X 方向可见 ±0.384 m	主场景必须用 Stage 沉浸场景，窗口放不下半径 1.6 m 的柱阵
顶点色	不可靠	一律烘焙进贴图，别依赖顶点色通道
方向光	对自定义网格无效	光照同样烘进贴图，别指望运行时打光

一句话判断

声之建筑是四方向里天花板最高、且唯一独占一条官方硬指标的一个。它的难处不在想法，在「能不能在真机上被听见」——所以整份方案的重量都压在 11.20 那次真机预约上。

放弃信号：11.22 仍无法在真机验证声源定位。此时切方向 D，不是因为 D 更好，而是因为**拿不到的设备不会因为再等一周就来**。

反过来说，**只要 11.20 能戴上头显听到「绕到柱背声音变闷」，这个方向就没有理由再犹豫** —— 那时候真正要花力气的是把它做得更好看、更完整，而不是再比较方向。

本方案书的依据与边界

- **已实测**：官方 SDK 6.1 agent-vault 文档逐条核对；BOM 6.1.9 依赖解析与构建通过（包体 +80 KB）；四窗口工程与 42 个内核单测在位（commit `97bc6ad` / `3b523a8`）。竞品信息来自公开飞书文档，其创意方向与排期可能随对方进展变化。
- **未实测**：真机上的空间音频听感、`stage.style` 数字枚举、手势手感 —— 三项均已列入 W1–W2 验证任务与止损线。
- **估算法**：六维预估 91 为上限值，真机未验前按 85–88 折算；18–22 人日不含稳定性压测与打磨。